

SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures

COURSE CODE: CSA03

Register Number	192572052
Name	SARANYA.R.A

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)*

Assessment Tool Weightage: 35%

Questions:2 EACH QUESTIONS CARRIES: 10 MARKS Total Marks:20 Duration: ONE HOUR

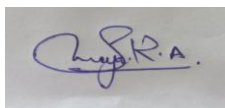
Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

I Saranya.R.A (192572052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

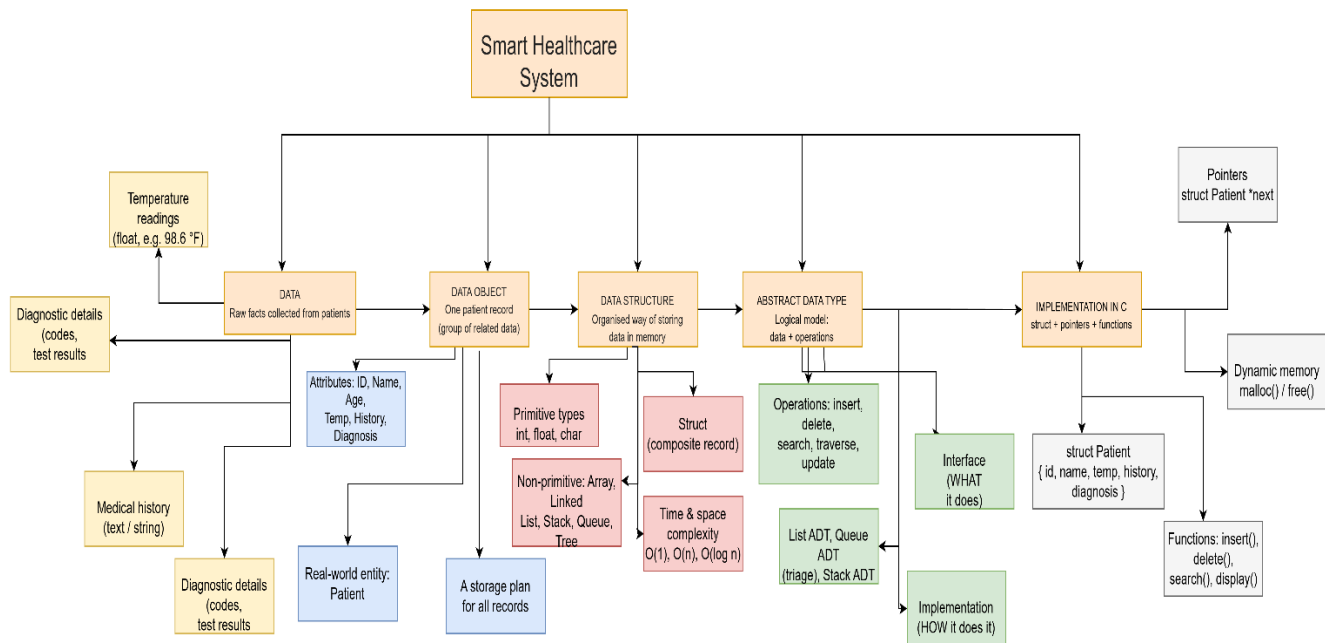
ANSWER:

1. Smart Healthcare System

The concept map on the next page moves from the raw facts the hospital collects (Data) down to the C code that stores and processes them (Implementation). Each level adds structure, so the system becomes faster, uses less memory and is easier to maintain. The relationships between the levels are: Data is grouped into a Data Object; a Data Object is organised using a Data Structure; a Data Structure is abstracted as an ADT; an ADT is realised through an Implementation in C.

Level-by-level justification

Level	Meaning	Healthcare example	Contribution to efficiency
1. Data	Raw, unprocessed facts with no structure.	Temperature 98.6 °F, 'diabetic since 2018', blood-test result.	Knowing the type (float, string, code) lets the system reserve the right amount of memory and avoid wasted space.
2. Data Object	A named group of related data items describing one entity.	Patient record: ID, name, age, temperature, history, diagnosis.	Keeps all facts about one patient together, so a record is stored, retrieved and updated as one unit; reduces duplication and errors.
3. Data Structure	A way of organising data objects in memory for efficient access.	Array of temperature readings; linked list of patients; queue for emergency triage.	The right structure reduces time and space complexity: array access $O(1)$, linked-list insertion $O(1)$, queue add/remove $O(1)$.
4. ADT	A logical model defined by its data and operations; the 'how' is hidden.	List ADT with insert, delete, search, traverse, update; Queue ADT for appointments.	Separates WHAT from HOW, so the storage can change (array → linked list) without changing the hospital application: modular, reusable, maintainable.
5. Implementation in C	Turning the ADT into working code.	struct Patient, pointer next, malloc()/free(), functions insert(), search(), display().	Pointers and dynamic memory let the record list grow as patients arrive and use memory only when needed; functions give fast, tested operations.



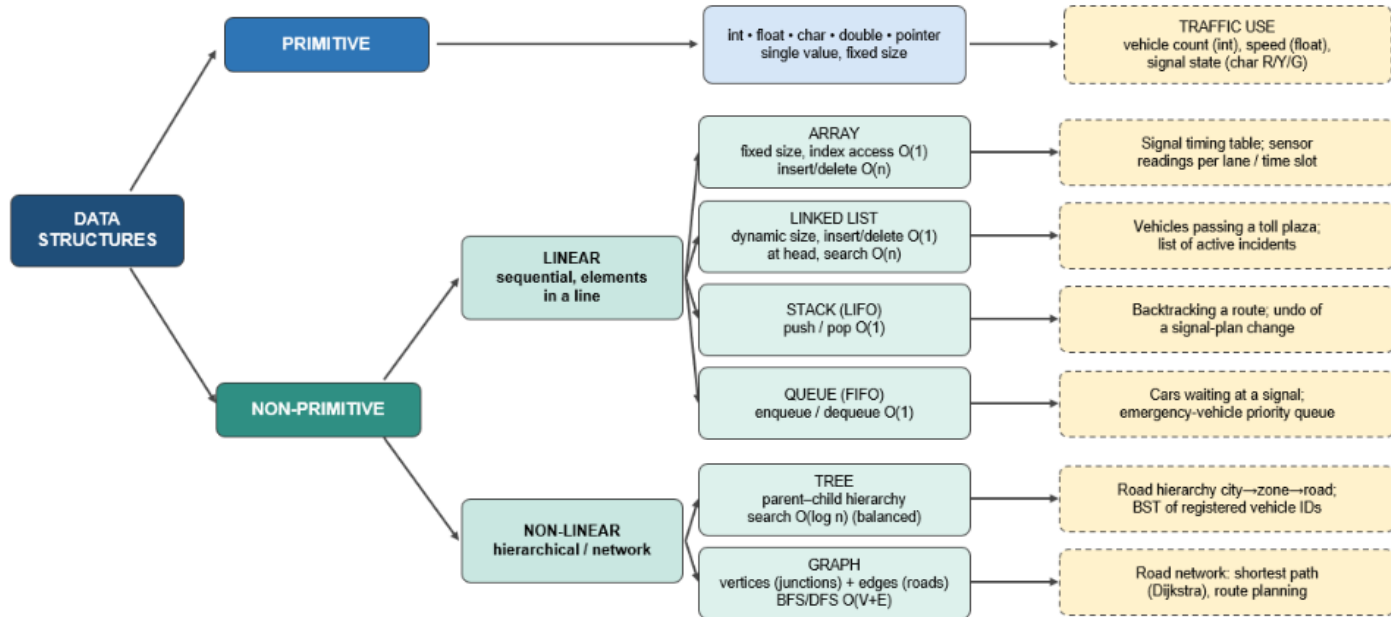
Sub-concepts and how they connect

- **Structures (struct):** combine different data types (int id, char name[], float temp) into one Patient record – this is the C form of a Data Object.
- **Pointers:** a pointer such as struct Patient *next links one record to the next, which forms a linked list and allows records to be added or removed without shifting data.
- **Operations:** insert, delete, search, traverse and update are the ADT interface; each is coded as a C function.
- **Complexity:** time and space complexity is the yardstick used at the Data Structure level to decide between array (fast access, fixed size) and linked list (flexible size, slower search).

conclusion

Each level removes a problem of the level above: raw data becomes meaningful records (Data Object), records are stored efficiently (Data Structure), the storage is hidden behind a clean interface (ADT), and pointers with dynamic memory make it work in C (Implementation). Together they give faster patient look-up, lower memory use and a system that can grow with the number of patients.

2. Smart City Traffic Management System



Classification and applications

Structure	Category	Traffic-management application	Efficiency	Why it is classified here
int, float, char, double, pointer	Primitive	Vehicle count (int), speed (float), signal state (char R/Y/G).	$O(1)$ direct CPU support.	Hold a single value of fixed size; they are the building blocks of everything else.
Array	Non-primitive, Linear	Signal timing table; sensor readings per lane and time slot.	Access $O(1)$; insert/delete $O(n)$.	Same-type elements in contiguous memory, one after another.
Linked list	Non-primitive, Linear	Vehicles passing a toll plaza; list of active incidents.	Insert/delete $O(1)$ at head; search $O(n)$.	Nodes connected by pointers in a single sequence; size changes at run time.
Stack (LIFO)	Non-primitive, Linear	Backtracking a route; undo of a signal-plan change.	Push/pop $O(1)$.	Linear structure with access only at one end (top).
Queue (FIFO)	Non-primitive, Linear	Vehicles waiting at a signal; emergency-vehicle priority queue.	Enqueue/dequeue $O(1)$.	Linear structure; insert at rear, remove at front.
Tree	Non-primitive, Non-linear	Road hierarchy (city → zone → road); BST of registered vehicle IDs.	Search $O(\log n)$ if balanced.	Parent-child hierarchy; one element can have many children.
Graph	Non-primitive, Non-linear	Road network (junctions = vertices, roads = edges); shortest path with Dijkstra; route planning.	Traversal $O(V + E)$.	Any element can connect to many others; models networks, even with cycles.

Relationships between the structures

- **Primitive → Non-primitive:** non-primitive structures are derived from primitive types. An array of int, or a node holding int data and a pointer, is built from primitives.
- **Non-primitive → Linear / Non-linear:** non-primitive structures are split by how elements are arranged. In linear structures each element has one predecessor and one successor and elements form a single sequence. In non-linear structures one element can link to many.
- **Within Linear:** stack and queue are restricted forms of the array or linked list that limit where insertion and removal happen (one end for a stack; opposite ends for a queue).
- **Linear → Non-linear:** a linked list node has one pointer; give a node two or more child pointers and you get a tree; allow arbitrary links, including cycles, and you get a graph. So tree and graph generalise the linked list.

Justification of the classification

By functionality: a traffic system needs different jobs done. Single readings such as speed or vehicle count need primitives. Ordered sequences such as signal timings or waiting vehicles need linear structures (array, queue). Hierarchies such as city → zone → road need a tree. A real road network, where a junction connects to many roads, can only be modelled naturally by a graph.

By efficiency: an array gives instant $O(1)$ access to a lane's sensor data but is costly to resize; a linked list grows easily as vehicles arrive; a queue handles first-come-first-served signal waiting in $O(1)$; a balanced tree searches vehicle records in $O(\log n)$ instead of $O(n)$; a graph with Dijkstra's algorithm finds the fastest route across the road network. Choosing the structure that matches the access pattern gives faster response and lower memory use.

Conclusion

Primitive types store single values; linear structures suit sequences (timings, queues); non-linear structures suit hierarchies and road networks. Matching the structure to the task gives the best functionality and efficiency.